Daniel Carvalho – a22209980

Guião 1 – Classes e Objectos

Objectivos:

- Definição de classes
  - Inicialização
  - Atributos
  - Métodos
- Instânciação de Objectos
  - id
  - Referências

1. Numa *interactive shell* Python 3 defina uma classe `Pessoa`.

```
>>> class Pessoa:
...     def __init__(self,nome,idade):
...         self.nome=nome
...         self.idade=idade
```

Aqui foi definida uma classe Pessoa

2. Instancie uma classe `Pessoa` e faça a atribuição a uma variável `pessoa_1`.

   Interprete o que vê quando executa:

   `>>>pessoa_1`

```
...
>>> pessoa_1=Pessoa("Antonio",23)
>>> pessoa_1
<__main__.Pessoa object at 0x71c23e849d80>
```

Significa que pessoa_1 é um objeto da classe Pessoa, criada no módulo principal (main) e com o endereço de memória 0x71c23e849d80

3. Verifique o id do objecto referênciado por `pessoa_1`.

```
>>> id(pessoa_1)
125079086472576
>>>
```

4. Compare os valores obtidos no ponto 2 e 3.

O valor obtido na questão 2 é a representação da instância, enquanto id(pessoa_1) é uma forma de identificá-la de maneira única na memória. Embora representem diferentes aspetos do objeto (um sendo o objeto em si e o outro seu identificador), eles referem-se ao mesmo objeto.

5. Repita o ponto 2 e reveja o id. O que alterou?

```
>>> pessoa_2 = Pessoa ("Mario",24)
>>> id(pessoa_2)
125079084080560
>>>
```

Criando uma nova instância, um novo objeto, o id será diferente.

6. A função `sys.getrefcount()` permite saber quantas referências estão activas para um determinado objecto. Veja quantas referências existem para o objecto referênciado pela variável `pessoa_1`. Comente o resultado.

O valor retornado é maior do que o esperado porque a função cria uma referência temporária.

7.

O objeto terá 3 referências ativas pessoa_1, “a” e dentro da lista no primeiro elemento da lista há uma referência ativa

8.

```python
a=1
b=1
print(a is b) # True
a=[1]
b=[1]
print(a is b) # False -- sao listas diferentes com o mesmo conteúdo. a e b nao referenciam o mesmo objeto
a=[1]
b=a
print(a is b)  # True --  b=a é dizer para b referenciar o mesmo objeto que a, entao a e b referenciam o mesmo objeto
a=[1]
b=a[:]
print(a is b)  # False -- b = a[:] cria uma copia da lista a, mas embora tenham o mesmo conteúdo, sao objetos diferentes
```

9 a 16

```python
class PizzMedia():
    #Classe para representar pizzas (médias)

    diametro_cm=30 #Atributo da classe que define um diametro default (30 centimetros)

    def __init__(self,nome, ingredientes= None): # Construtor que inicializa uma instancia da classe PizzMedia
        self.nome=nome #Atributo de instancia que armazena o nome da pizza
        self.ingredientes = ingredientes if ingredientes is not None else ['tomate','mozarella']
        # Atributo de instancia que armaneza a lista de ingredientes
        # Se o valor de ingredientes for None, será atribuida a lista padrao ['tomate' , 'mozarella´] como ingredientes

    1 usage
    def nome_nome_maisculas(self):
        #Método que retorna o nome em maiusculas
        return self.nome.upper()

    1 usage
    def area (self):
        #Método que retorna a area da pizza
        return 3.14 * (self.diametro_cm/2) **2
```

```python
# Criar uma pizza com ingredientes personalizados
pizza1 = PizzMedia(nome="Personalizada", ingredientes=['bacon', 'tomate','parmesão', 'fiambre'])

#Prints

print("Pizza 1:")
print("Nome:", pizza1.nome)
print("Nome em maiúsculas:", pizza1.nome_nome_maisculas())
print("Ingredientes:", pizza1.ingredientes)
print("Área da pizza:", pizza1.area(), "cm quadrados")
```